

自注意力和位置编码

自注意力让序列中的每个词元直接关注所有词元，能够并行计算，也缩短了远距离信息之间的传递路径。但它的计算量随序列长度的平方增长，处理很长的序列时会非常慢。为了让模型利用词元的先后顺序，可以在输入表示中加入位置编码。

自注意力如何表示序列

卷积神经网络（CNN）和循环神经网络（RNN）常用于序列编码。注意力机制提供了另一种方式：让同一组输入同时充当查询、键和值，每个查询关注所有键值对，并生成一个输出。这种机制称为**自注意力**（self-attention；Lin、Feng、Santos 等，2017；Vaswani、Shazeer、Parmar 等，2017），也称为**内部注意力**（intra-attention；Cheng、Dong、Lapata，2016；Parikh、Tackstrom、Das 等，2016；Paulus、Xiong、Socher，2017）。

设输入序列为 $\mathbf{x}_1, \dots, \mathbf{x}_n$ ，其中 $\mathbf{x}_i \in \mathbb{R}^d$ ， $1 \leq i \leq n$ 。自注意力生成等长的输出序列 $\mathbf{y}_1, \dots, \mathbf{y}_n$ ：

$$\mathbf{y}_i = f(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \dots, (\mathbf{x}_n, \mathbf{x}_n)) \in \mathbb{R}^d$$

这里， f 是注意力汇聚函数。它以 $\mathbf{x}_i$ 为查询，以全部输入组成键值对，得到该位置的输出。

实现时，可以把同一个张量分别作为多头注意力的查询、键和值。张量形状为“批量大小 $\times$ 序列长度 $\times$ 隐藏维度”，输出形状与输入相同。例如，设置隐藏维度为 100、注意力头数为 5、暂退率为 0.5，使用形状为 (2, 4, 100) 的全一输入，并将两个样本的有效长度设为 3 和 2，输出形状仍为 (2, 4, 100)。

CNN、RNN 与自注意力的比较

比较这三种架构时，设它们都将长度为 n 的序列映射为等长序列，每个输入和输出词元都用 d 维向量表示。除了计算复杂度，还需要考察顺序操作的数量和最大路径长度：顺序操作限制并行计算；序列位置之间的路径越短，就越容易学习远距离依赖关系（Hochreiter、Bengio、Frasconi 等，2001）。

架构	计算复杂度	顺序操作数量	最大路径长度
CNN（卷积核大小为 k ）	$\mathcal{O}(knd^2)$	$\mathcal{O}(1)$	$\mathcal{O}(n/k)$
RNN	$\mathcal{O}(nd^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
自注意力	$\mathcal{O}(n^2d)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

CNN 通过局部卷积逐层连接不同位置。当卷积核大小为 k 、输入和输出通道数均为 d 时，长度为 n 的序列需要 $\mathcal{O}(knd^2)$ 的计算量。卷积层内的不同位置可以并行计算，顺序操作数量为 $\mathcal{O}(1)$ ；跨位置传递信息的最大路径长度为 $\mathcal{O}(n/k)$ 。例如，卷积核大小为 3 的两层卷积网络，其感受野可以同时包含 $\mathbf{x}_1$ 和 $\mathbf{x}_5$ 。

RNN 需要沿序列依次更新隐状态。每一步中， $d \times d$ 的权重矩阵与 d 维隐状态相乘，计算复杂度为 $\mathcal{O}(d^2)$ ；处理 n 步的总复杂度为 $\mathcal{O}(nd^2)$ 。这些更新中有 $\mathcal{O}(n)$ 个顺序操作无法并行化，最大路径长度也为 $\mathcal{O}(n)$ 。

自注意力直接连接任意两个词元。查询、键和值都是 $n \times d$ 矩阵。在缩放点积注意力中，先将 $n \times d$ 的查询矩阵与 $d \times n$ 的键转置矩阵相乘，得到 $n \times n$ 矩阵；再由注意力权重矩阵与 $n \times d$ 的值矩阵相乘，得到输出。因此，计算复杂度为 $\mathcal{O}(n^2d)$ 。不同词元的计算可以并行进行，顺序操作数量和最大路径长度均为 $\mathcal{O}(1)$ 。

CNN 和自注意力都具有并行计算的优势，自注意力的最大路径长度最短。不过，自注意力的计算复杂度对序列长度呈二次增长，这也使它在很长的序列上计算缓慢。

用位置编码补充顺序信息

RNN 逐个处理词元，自注意力则通过并行计算省去了这种顺序操作。要让自注意力利用序列顺序，可以在输入表示中加入**位置编码**（positional encoding），注入绝对或相对位置信息。位置编码既可以通过学习得到，也可以预先固定。下面采用 Vaswani、Shazeer、Parmar 等（2017）提出的固定正弦、余弦位置编码。

设 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 包含 n 个词元的 d 维嵌入，位置编码矩阵 $\mathbf{P} \in \mathbb{R}^{n \times d}$ 与它形状相同。加入位置编码后的表示为 $\mathbf{X} + \mathbf{P}$ 。按从 0 开始的索引， $\mathbf{P}$ 第 i 行、第 $2j$ 列和第 $2j+1$ 列的元素分别为：

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right),$$
$$p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right).$$

矩阵的行对应词元位置，列对应编码维度。相邻的一对列分别使用正弦和余弦；随着编码维度增加，频率降低。例如，取编码维度为 32、序列长度为 60，观察索引为 6、7、8、9 的四列：第 6、7 列的频率高于第 8、9 列；第 6 列与第 7 列之间，以及第 8 列与第 9 列之间的相位偏移，来自正弦和余弦的交替使用。

实现位置编码时，可以先按公式生成一个足够长的矩阵。例如，预设最大长度为 1000，存储形状为 $(1, 1000, d)$ 的编码；每次按实际序列长度取出对应部分，加到输入上，再应用暂退法。在 PyTorch 中，相加前还需将位置编码放到输入所在的设备上。若要单独观察编码，可令输入全为零、暂退率为 0；上述 32 维、60 步的示例便采用这种方式。

绝对位置信息：不同维度以不同频率变化

二进制表示可以帮助理解这种频率设计。整数 0 到 7 对应的三位二进制依次为：000、001、010、011、100、101、110、111。最低位每隔 1 个数翻转，第二个最低位每隔 2 个数翻转，第三个最低位每隔 4 个数翻转。也就是说，较高位的变化频率低于较低位。

正弦、余弦位置编码也采用类似的频率变化方式，只是用三角函数让频率沿编码维度降低。它的输出是浮点数，这种连续表示比二进制表示更节省空间。

相对位置信息：固定偏移对应同一个线性变换

这种位置编码不仅能表示绝对位置，还允许模型学习相对位置信息。关键在于：对于固定的位置偏移 δ ，位置 $i + \delta$ 的编码可以由位置 i 的编码通过线性变换得到。

令 $\omega_j = 1/10000^{2j/d}$ 。对任意一对编码分量，有：

$$\begin{aligned} & \begin{bmatrix} \cos(\delta\omega_j) & \sin(\delta\omega_j) \\ -\sin(\delta\omega_j) & \cos(\delta\omega_j) \end{bmatrix} \begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix} \\ &= \begin{bmatrix} \cos(\delta\omega_j)\sin(i\omega_j) + \sin(\delta\omega_j)\cos(i\omega_j) \\ -\sin(\delta\omega_j)\sin(i\omega_j) + \cos(\delta\omega_j)\cos(i\omega_j) \end{bmatrix} \\ &= \begin{bmatrix} \sin((i+\delta)\omega_j) \\ \cos((i+\delta)\omega_j) \end{bmatrix} \\ &= \begin{bmatrix} p_{i+\delta,2j} \\ p_{i+\delta,2j+1} \end{bmatrix}. \end{aligned}$$

这里的 2×2 投影矩阵不依赖位置索引 i 。因此，对于给定的编码分量对，同一个偏移 δ 在不同位置都对应同一个线性变换，这使模型能够学习相对位置信息。

小结

- 自注意力的查询、键和值来自同一组输入，每个词元都可以关注序列中的所有词元。
- CNN 和自注意力都便于并行计算，自注意力的最大路径长度最短。但自注意力的计算量随序列长度的平方增长，在很长的序列上计算会非常慢。
- 在输入表示中加入位置编码，可以补充序列的顺序信息，注入绝对或相对位置信息。

练习

1. 假设通过堆叠带有位置编码的自注意力层，构建一个表示序列的深度架构，可能会遇到什么问题？
2. 设计一种可以通过学习得到的位置编码方法。

讨论区：[MXNet](#)、[PyTorch](#)、[Paddle](#)。